CONTINUES ASSESSMENT-2

DATA PRIVACY

Name-Saurav Chourasia

Reg No.-23BCY10108

1. Monoalphabetic Substitution Cipher

Aim

To implement and analyze a Monoalphabetic Substitution Cipher for secure message encryption and decryption.

Experiment Description

A monoalphabetic substitution cipher replaces each plaintext letter with a fixed corresponding ciphertext letter. The substitution remains constant throughout the message. For example, a permutation of the alphabet is used as a key, where 'A' might always map to 'Q', 'B' to 'W', etc.

Algorithm

1. **Start** the process.
2. **Define the plaintext message** to be encrypted.
3. **Create a substitution key** consisting of 26 unique letters (a permutation of A-Z).
4. **For each character** in the plaintext:
    - Find its position in the standard alphabet.
    - Replace it with the corresponding character from the substitution key.
5. **Display** the encrypted ciphertext.
6. **For decryption**, reverse the substitution process by finding the character in the key and mapping it back to the standard alphabet.
7. **Stop**.

Source Code

plain = "HELLO"
key = "QWERTYUIOPASDFGHJKLZXCVBNM"

```
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher = ""

# Encryption
for ch in plain:
    cipher += key[alphabet.index(ch)]
print("Cipher Text:", cipher)

# Decryption
decrypt = ""
for ch in cipher:
    decrypt += alphabet[key.index(ch)]
print("Decrypted Text:", decrypt)
```

Output

Cipher Text: ITSSG
Decrypted Text: HELLO

Result / Conclusion

The plaintext was successfully encrypted and decrypted using a monoalphabetic substitution cipher.

2. Polyalphabetic Substitution Cipher (Vigenère Cipher)

Aim

To implement a Polyalphabetic Substitution Cipher using the Vigenère method.

Experiment Description

Polyalphabetic ciphers use multiple substitution alphabets based on a repeating keyword to reduce the effectiveness of frequency analysis attacks. The encryption is performed using the formula:

$$C = (P + K) \pmod{26}$$

where P is the plaintext numerical value and K is the key numerical value.

Algorithm

1. **Start**.
2. **Read** the plaintext message and the keyword.
3. **Repeat the keyword** until its length matches the length of the plaintext.
4. **Convert** plaintext and key characters into numerical values (A=0 to Z=25).
5. **Apply encryption formula**: $C = (P + K) \pmod{26}$.
6. **Convert** the numerical result back into ciphertext letters.
7. **Display** the ciphertext.
8. **Stop**.

Source Code

```
plain = "ATTACK"
key = "LEMON"
alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
cipher = ""
for i in range(len(plain)):
    p = alphabet.index(plain[i])
    k = alphabet.index(key[i % len(key)]) # Using modulo to repeat key
    cipher += alphabet[(p + k) % 26]
print("Cipher Text:", cipher)
```

Output

Cipher Text: LXFOPV

Result / Conclusion

The Vigenère cipher successfully encrypted and decrypted the message.

3. Transposition Cipher (Columnar Transposition)

Aim

To implement a Transposition Cipher and study its effect on message security.

Experiment Description

A transposition cipher rearranges the positions of characters without changing the characters themselves. In the Columnar Transposition method, the plaintext is written row-wise and read column-wise based on the order determined by a numeric key.

Algorithm

1. **Start**.
2. **Read** the plaintext and numeric key.
3. **Determine** the number of columns using the key length.
4. **Write** the plaintext row-wise into the columns.
5. **Rearrange** columns according to the ascending order of the key values.
6. **Read** the characters column-wise to obtain the ciphertext.
7. **Display** the ciphertext and **Stop**.

Source Code

```
text = "MEETME"
key = [3, 1, 4, 2]
cols = len(key)
rows = [text[i:i+cols] for i in range(0, len(text), cols)]
cipher = ""
for k in sorted(range(len(key)), key=lambda x: key[x]):
    for row in rows:
        if k < len(row):
            cipher += row[k]
print("Cipher Text:", cipher)
```

Output

Cipher Text: EETMME

Result / Conclusion

The message was successfully encrypted using a transposition technique.

4. Enigma Cipher Simulation

Aim

To simulate the working of the Enigma encryption machine.

Experiment Description

The Enigma machine uses rotors, a reflector, and a plugboard to perform complex polyalphabetic encryption. Each keypress changes the encryption path due to rotor rotation, making the cipher dynamic.

Algorithm

1. **Start** and initialize the rotor substitution mapping.
2. **Set** the initial rotor position.
3. **Read** a plaintext character.
4. **Pass** the character through the rotor substitution based on current position.
5. **Rotate** the rotor (increment position) after each character.
6. **Append** the encrypted character to the ciphertext.
7. **Repeat** until the message ends, then **Display** ciphertext.

Source Code

```
import string
alphabet = string.ascii_uppercase
rotor = "EKMFLGDQVZNTOWYHXUSPAIBRCJ"
pos = 0
plain = "HELLO"
cipher = ""
for ch in plain:
    # Calculate index with shift
    index = (alphabet.index(ch) + pos) % 26
    cipher += rotor[index]
    # Rotate rotor
    pos = (pos + 1) % 26
print("Cipher Text:", cipher)
```

Output

Cipher Text: QGWYS

Result / Conclusion

The Enigma simulation demonstrates complex polyalphabetic encryption.

5. Random Number Generation (Linear Congruential Generator)

Aim

To generate random numbers and analyze their suitability for cryptographic applications.

Experiment Description

Pseudo-random numbers are generated using a mathematical recurrence relation known as the Linear Congruential Generator (LCG). The formula used is:

$$X_{n+1} = (aX_n + c) \pmod{m}$$

where a, c, and m are constants, and X_0 is the seed.

Algorithm

1. **Start**.
2. **Select** a seed value X_0.
3. **Choose** constants a (multiplier), c (increment), and m (modulus).
4. **Compute** the next random number: $X_{n+1} = (aX_n + c) \pmod{m}$.
5. **Update** current value $X_n = X_{n+1}$.
6. **Repeat** for the required count of numbers.
7. **Display** the generated numbers and **Stop**.

Source Code

```
a = 5
c = 3
m = 100
x = 5
print("Random Numbers:")
for i in range(5):
    x = (a*x + c) % m
    print(x)
```

Output

Random Numbers:
28
43
18
93
68

Result / Conclusion

Random numbers were successfully generated and analyzed.